❹ UNIDEV™ (suite)

Aller sur Moodle et copier le fichier « Unidev.sql » qui vous permettra de générer les tables de la base de données et d'insérer quelques tuples dans ces tables.

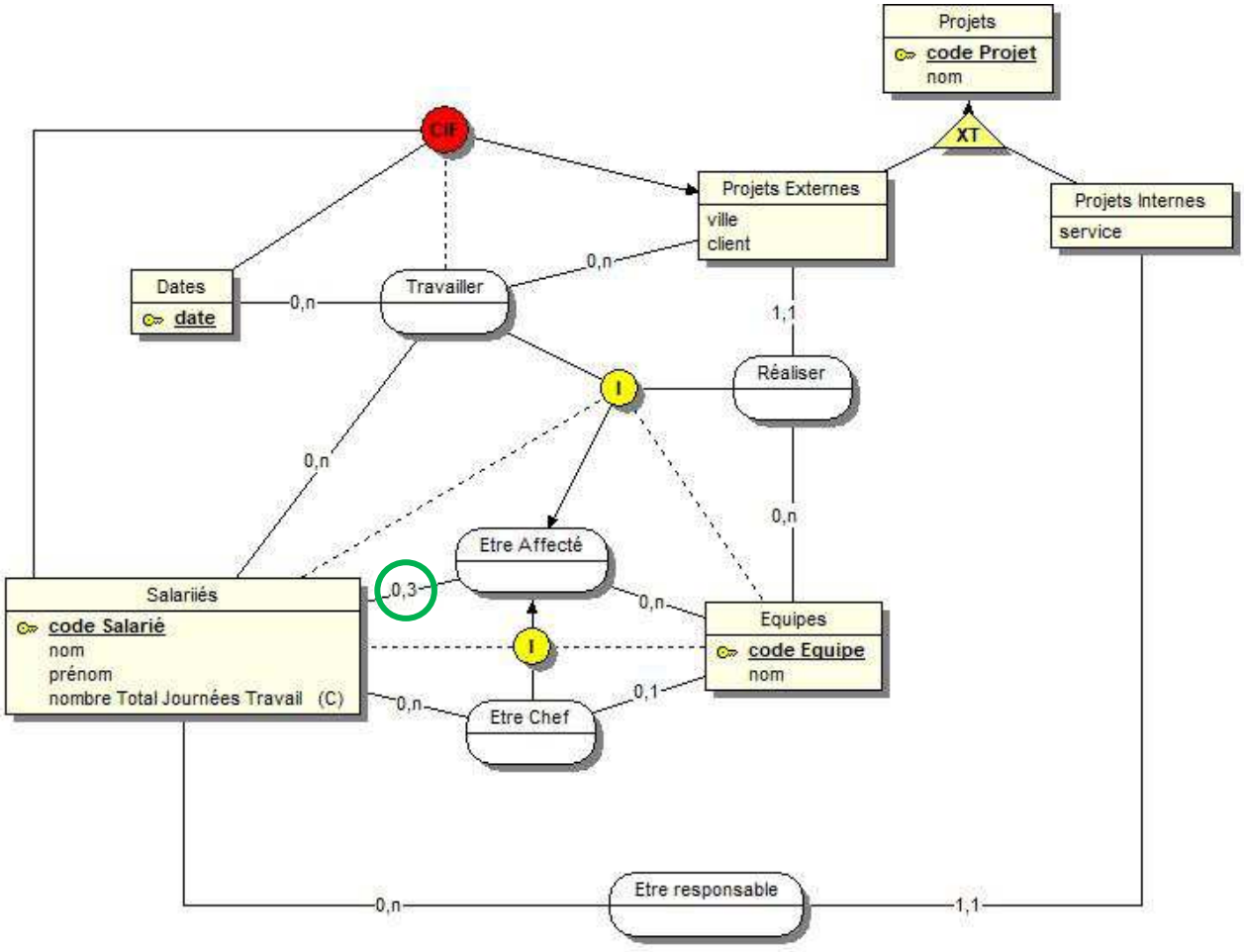
Vous allez ainsi obtenir une base de données correspondant au schéma relationnel suivant :

SALARIES (codeSalarie, nomSalarie, prenomSalarie, *nbTotalJourneesTravail*)
EQUIPES (codeEquipe, nomEquipe, codeSalarieChef#)
PROJETSEXTERNES (codeProjet, nomProjet, villeProjet, clientProjet, codeEquipe#)
PROJETSINTERNES (codeProjet, nomProjet, serviceProjet, codeSalarieResponsable#)
ETREAFFECTE (codeSalarie#, codeEquipe#)
TRAVAILLER (codeSalarie#, dateTravail, codeProjet#)

Dans la relation *Salaries,* l'attribut `nbTotalJourneesTravail` (qui est indiqué en italique) est calculable à partir d'une requête SQL. Il engendre donc de la redondance dans la base de données, mais il a tout de même été maintenu car il permet d'accélérer certaines requêtes qui sont fréquemment exécutées.

Attention, cette base de données ne respecte pas les contraintes A, B, C, D et E vues à la question précédente.
Dans le TP qui va suivre, vous allez donc implémenter ces contraintes soit dans le Langage de Définition de Données SQL quand cela est possible, soit en PL/SQL dans les cas contraires (avec des procédures stockées ou bien des triggers).
Pour rappel, vous trouverez ci-dessous le modèle entités-associations avec toutes les contraintes à prendre en compte.

Il faudra aussi s'assurer que l'attribut calculé *nbTotalJourneesTravail* de la table `Salaries` soit toujours à jour, même lorsque on modifie les données dans la table `Travailler`.



Deuxième partie – Gestion des contraintes du schéma relationnel avec le LDD SQL :

5) Contraintes d'unicité ou CIF (Contrainte d'Intégrité Fonctionnelle) sur une ternaire.
   - On souhaite modifier la base de données afin qu'un salarié ne puisse pas travailler sur plusieurs projets à une même date ; et ce sans écrire de code PL/SQL. On vous rappelle que l'instruction SQL `ALTER TABLE` permet de modifier une table. Et qu'avec cette instruction il est possible de supprimer une contrainte (`ALTER TABLE maTable DROP CONSTRAINT ...`) ou bien de rajouter une contrainte (`ALTER TABLE maTable ADD CONSTRAINT ...`).
   - Résultat attendu :

```
SELECT dateTravail, codeProjet
FROM Travailler
WHERE codeSalarie = 'S6';

DATETRAVAIL  CODEPROJET
------------ -----------
02/01/22     P4
03/01/22     P4
08/01/22     P5
```

   L'instruction suivante doit déclencher **une erreur** (le salarié S6 travaille déjà sur un projet le 8 janvier)

```
INSERT INTO Travailler (codeSalarie, dateTravail, codeProjet)
                VALUES ('S6', '08/01/2022', 'P4') ;
```

6) Contraintes d'inclusion entre deux associations.
   - On souhaite programmer la contrainte qui empêche un salarié d'être chef d'une équipe où il n'est pas affecté, notamment lorsqu'on modifie le chef d'une équipe qui existe déjà (mais il faudrait que cette contrainte soit aussi respectée quand on crée une nouvelle équipe ou lorsqu'on modifie ou supprime des affectations existantes). Ici aussi, on ne veut pas écrire de code PL/SQL.
   - Résultat attendu :

```
SELECT codeEquipe
FROM EtreAffecte
WHERE codeSalarie = 'S3';

CODEEQUIPE
-----------
E3
E4
```

   Si on indique que le salarié S3 est chef de l'équipe E2 il doit y avoir **une erreur** (car le salarié S3 n'est pas affecté à l'équipe E2 ; il ne peut donc pas en être le chef).

```
UPDATE Equipes
SET codeSalarieChef = 'S3'
WHERE codeEquipe = 'E2';
```

   Par contre, si on indique que le salarié S3 est chef de l'équipe E4 il ne doit pas y avoir d'erreur car il est affecté à cette équipe.

```
UPDATE Equipes
SET codeSalarieChef = 'S3'
WHERE codeEquipe = 'E4';
```

Troisième partie – Gestion des contraintes avec des procédures stockées PL/SQL :

7) Mise à jour automatique d'un attribut calculé.
   - On souhaite que l'attribut `nbTotalJourneesTravail` de la table `Salaries` soit mis à jour automatiquement lorsque on insère une nouvelle ligne dans la table `Travailler` (pour simplifier, on ne gèrera pas encore les cas où on modifie ou supprime des lignes de cette table).

     Pour cela, au lieu de faire directement des insertions dans la table `Travailler`, on utilisera une procédure `AjouterJourneeTravail` que l'on vous demande d'écrire et qui doit avoir la signature suivante :

```
PROCEDURE AjouterJourneeTravail (
                        p_codeSalarie Travailler.codeSalarie%TYPE,
                          p_codeProjet Travailler.codeProjet%TYPE,
                        p_dateTravail Travailler.dateTravail%TYPE)
```

- Résultat attendu :

```
SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S2';

NBTOTALJOURNEESTRAVAIL
----------------------
7
```

L'ajout d'une journée de travail pour le salarié S2 doit incrémenter de 1 son nombre total de journées travaillées.

```
CALL AjouterJourneeTravail('S2','P3', '10/01/2022');

SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S2';

NBTOTALJOURNEESTRAVAIL
----------------------
8
```

8) Gestion des multiplicités maximales des associations.

- On souhaite maintenant programmer la contrainte qui empêche un salarié d'être affecté à plus de trois équipes lorsqu'on insère une nouvelle ligne dans la table `EtreAffecte` (pour simplifier on ne gèrera pas encore les cas où on modifie des lignes de cette table).

  Pour cela, au lieu de faire directement des insertions dans la table `EtreAffecte`, on utilisera une procédure `AffecterSalarieEquipe` que l'on vous demande d'écrire et dont la signature doit être la suivante :

```
PROCEDURE AffecterSalarieEquipe (
                    p_codeSalarie EtreAffecte.codeSalarie%TYPE,
                      p_codeEquipe EtreAffecte.codeEquipe%TYPE)
```

  Si le salarié dont le code est passé en paramètre n'a pas déjà été affecté à trois équipes, cette procédure l'affecte à l'équipe qui est passée en paramètre. Par contre, s'il est déjà affecté à trois équipes, cette procédure lève une exception avec l'instruction suivante :

```
RAISE_APPLICATION_ERROR(-20001, 'Le salarié est déjà affecté à
                                          au moins 3 équipes');
```

- Résultat attendu :

```
SELECT codeSalarie, codeEquipe
FROM EtreAffecte
WHERE codeSalarie IN ('S1', 'S8');

CODESALARIE  CODEEQUIPE
------------ -----------
S1           E1
S1           E2
S8           E2
S8           E3
S8           E4
```

L'instruction suivante doit déclencher **une erreur** car le salarié S8 est déjà affecté à 3 équipes.

```
CALL AffecterSalarieEquipe('S8', 'E1');
Le salarié est déjà affecté à au moins 3 équipes

SELECT *
FROM EtreAffecte
WHERE codeSalarie = 'S8'
AND codeEquipe = 'E1';

aucune ligne sélectionnée
```

Par contre, l'instruction suivante ne déclenchera pas d'erreur et va rajouter une ligne dans la table `EtreAffecte`.

```
CALL AffecterSalarieEquipe('S1', 'E3');

SELECT *
FROM EtreAffecte
WHERE codeSalarie = 'S1'
AND codeEquipe = 'E3';

CODESALARIE  CODEEQUIPE
------------ -----------
S1           E3
```

9) Gestion des contraintes sur un héritage.

- On souhaite maintenant programmer la contrainte de partition (XT) sur l'héritage entre `Projets`, `ProjetsExternes` et `ProjetsInternes`. La décomposition descendante a permis d'empêcher d'avoir des projets qui ne sont ni internes, ni externes. Mais il faut encore empêcher d'avoir des projets qui sont à la fois internes et externes lorsqu'on insère une nouvelle ligne dans la table `ProjetsExternes` ou dans la table `ProjetsInternes`.

  Pour cela, au lieu de faire directement des insertions dans ces tables on utilisera les procédures `AjouterProjetExterne` et `AjouterProjetInterne` que l'on vous demande d'écrire :

- Résultat attendu :

```
SELECT codeProjet, 'EXTERNE' AS typeProjet
FROM ProjetsExternes
UNION
SELECT codeProjet, 'INTERNE' AS typeProjet
FROM ProjetsInternes;

CODEPROJET  TYPEPROJET
----------- -----------
P1          EXTERNE
P2          EXTERNE
P3          EXTERNE
P4          EXTERNE
P5          EXTERNE
P6          INTERNE
P7          INTERNE
```

L'utilisation de la procédure `AjouterProjetExterne` pour ajouter un projet externe avec le code P7 doit déclencher **une erreur** car le projet P7 existe déjà dans la table `ProjetsInternes`. Ensuite si vous lancez la requête suivante, vous devriez obtenir ce résultat.

```
SELECT codeProjet, 'EXTERNE' AS typeProjet
FROM ProjetsExternes;

CODEPROJET  TYPEPROJET
----------- -----------
P1          EXTERNE
P2          EXTERNE
P3          EXTERNE
P4          EXTERNE
P5          EXTERNE
```

De même, l'utilisation de la procédure `AjouterProjetInterne` pour ajouter un projet interne P1 doit déclencher **une erreur** car le projet P1 existe déjà dans la table `ProjetsExternes`. Ensuite si vous lancez la requête suivante, vous devriez obtenir ce résultat.

```
SELECT codeProjet, 'INTERNE' AS typeProjet
FROM ProjetsInternes;

CODEPROJET  TYPEPROJET
----------- -----------
P6          INTERNE
P7          INTERNE
```

Par contre, l'utilisation de la procédure `AjouterProjetInterne` pour ajouter un projet interne P8 ayant pour nom 'Programme de comptabilité', comme service 'Service COMPTA' et comme responsable le salarié 'S9' ne déclenchera pas d'erreur et rajoutera une ligne dans la table `ProjetsInternes`. Ensuite si vous lancez la requête suivante, vous devriez obtenir ce résultat.

```
SELECT codeProjet, 'INTERNE' AS typeProjet
FROM ProjetsInternes;

CODEPROJET  TYPEPROJET
----------- -----------
P6          INTERNE
P7          INTERNE
P8          INTERNE
```

10) Contraintes d'inclusion concernant plusieurs associations.

- On souhaite programmer la contrainte qui empêche un salarié de travailler sur un projet qui est réalisé par une équipe dans laquelle n'est pas affecté le salarié en question. On veut que cette contrainte soit vérifiée à chaque fois que l'on ajoute une journée de travail d'un salarié (pour le moment, on ne souhaite pas gérer le cas où on modifie une journée de travail déjà existante).

- Résultat attendu :

```
SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S2';

NBTOTALJOURNEESTRAVAIL
----------------------
8
```

Si on indique que le salarié S2 a travaillé le 11/01/2022 sur le projet P3, il ne doit pas y avoir d'erreur. Ensuite, si on exécute la requête suivante, on obtiendra :

```
SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S2';

NBTOTALJOURNEESTRAVAIL
----------------------
9
```

Puis, si on indique que le salarié S2 a travaillé le 12/01/2022 sur le projet P5, il doit y avoir **une erreur** (car le salarié S2 n'est pas affecté à l'équipe qui travaille sur le projet P5). Ensuite, si on exécute la requête suivante, on obtiendra :

```
SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S2';

NBTOTALJOURNEESTRAVAIL
----------------------
9
```